# Galaxy Morphology Classifiers on RISC-V

Energy and Performance: CNN versus S4D with Vector Extensions

Measured on gem5 (RiscvMinorCPU) and McPAT, one inference per run

August 2026

## 1 What we did

We compare two galaxy morphology classifiers written from scratch in C for RISC-V and hand optimized with the RISC-V Vector extension (RVV 1.0). The first is a small convolutional network, the CNN only Large model with about 61 thousand parameters. The second is a state space model, S4D, in two sizes named d64 and d108. Both take one RGB image of size 64 by 64 and output one of four galaxy classes.

For each model we build two binaries from the same source, one that uses the vector unit and one scalar build with the vector paths disabled. We run one forward pass through the cycle level simulator gem5 using an in order core, then feed the resulting activity into the McPAT power model. The product of runtime dynamic power and execution time gives the energy per inference. This is the same pipeline and the same assumptions we used for the earlier S4D study, so the two sets of numbers are directly comparable.

## 2 Setup

Every run uses the same core so the hardware is held fixed and only the software changes. The core is a RiscvMinorCPU, in order, at 100 MHz. The vector unit is RVV 1.0 with a vector length of 256 bits. The McPAT model uses a 22 nm process and a floating point instruction fraction of 0.60, which matches the float heavy nature of both workloads. Each core has a private 16 KiB L1 instruction cache and a 16 KiB L1 data cache, with no L2 or L3. Instruction counts are gem5 committed (retired) instructions. The fixed hardware is an area of 1.567 mm squared, a peak dynamic power of 0.0591 W and a total leakage of 0.00157 W.

## 3 Results

Table 1 lists the measured numbers for every build. Table 2 shows how much the vector build beats the scalar build for each model. Table 3 places the CNN next to the best S4D model, d108, with both using the vector unit.

Table 1: Measured results, one inference, vector length 256, fp fraction 0.60

| Model | Build | Committed instr | Time (s) | Power (W) | Energy (mJ) |
|---|---|---|---|---|---|
| Galaxy CNN 61K | RVV | 53,825,016 | 0.8221 | 0.010009 | 8.228 |
| Galaxy CNN 61K | Scalar | 318,995,445 | 10.5470 | 0.005801 | 61.183 |
| S4D d108 | RVV | 19,948,328 | 0.2865 | 0.009935 | 2.846 |
| S4D d108 | Scalar | 33,051,024 | 0.5319 | 0.008931 | 4.750 |
| S4D d64 | RVV | 33,082,055 | 0.3827 | 0.010952 | 4.191 |
| S4D d64 | Scalar | 152,657,262 | 2.4981 | 0.008756 | 21.874 |

Table 2: How much RVV beats scalar (scalar value divided by RVV value)

| Model | Fewer instructions | Faster (time) | Less energy |
|---|---|---|---|
| Galaxy CNN 61K | 5.93x | 12.83x | 7.44x |
| S4D d108 | 1.66x | 1.86x | 1.67x |
| S4D d64 | 4.61x | 6.53x | 5.22x |

Table 3: CNN versus S4D d108, both vector builds, same core

| Metric | Galaxy CNN 61K | S4D d108 | CNN vs S4D |
|---|---|---|---|
| Committed instructions | 53,825,016 | 19,948,328 | 2.70x |
| Execution time (s) | 0.8221 | 0.2865 | 2.87x |
| Energy per inference (mJ) | 8.228 | 2.846 | 2.89x |
| Throughput (inferences per s) | 1.216 | 3.491 | 0.35x |
| Energy delay product (mJ times s) | 6.765 | 0.815 | 8.30x |
| Accuracy on GalaxyMNIST | not yet measured | 82.2% | |

## 4 Controlled experiment: fixed sequence length

The comparison above pairs models that differ in three ways at once, so to isolate model size from sequence length we retrained both S4D widths at the same 256 token sequence (seed 30485, same GalaxyMNIST). Table 4 holds the sequence fixed, so the only variables are width and depth.

Table 4: d64 vs d108 at a fixed 256 token sequence, RVV builds

| Metric | d64_seq256 (64w, 2L) | d108_seq256 (108w, 3L) | d108 / d64 |
|---|---|---|---|
| Committed instructions | 6,268,591 | 19,948,310 | 3.18x |
| Execution time (s) | 0.0834 | 0.2865 | 3.44x |
| Energy per inference (mJ) | 0.931 | 3.010 | 3.23x |
| Accuracy | 79.60% | 80.85% | +1.25 pts |

With the sequence length held constant, the larger model costs 3.2 times the instructions and 3.2 times the energy for 1.25 points of accuracy. This confirms that the earlier advantage of d108 over the native d64 came from its shorter 256 token sequence, not from its width. Sequence length is the dominant cost driver in S4D, and width is a second order effect paid for a small accuracy gain.

## 5 What the numbers say

Two clear results come out of the comparison.

First, S4D is the more energy efficient architecture for this task. On the same core, the CNN needs about 2.9 times the energy of S4D d108 per inference, along with 2.7 times the instructions and 2.9 times the runtime. Its energy delay product, which rewards being both fast and low energy, is more than eight times larger.

Second, the vector unit pays off much more for the CNN than for S4D. On the CNN, RVV cuts energy by 7.44 times against its own scalar build. On S4D d108 the same vector unit cuts energy by only 1.67 times. The reason is the shape of the work. The CNN convolution is expressed as an im2col transform followed by a general matrix multiply, which lays out many independent patches side by side, so the vector unit stays full. The S4D core is a recurrent scan where each step depends on the previous one, so there is less independent work for the vector lanes and part of the time is spent on cross lane reductions. The larger d108 also leaves less room for RVV to help because its scalar build already benefits from the compiler widening its projection layer.

Put together, the CNN is the heavier model per inference, but it is the one that rewards vector hardware the most. S4D remains the better choice when energy per inference is the priority.

## 6 Conclusion

Both classifiers run correctly from hand written vectorized C on a RISC-V in order core, and both are measured end to end for energy. S4D d108 is the most energy efficient at 2.846 mJ per inference with RVV, and it reaches 82.2 percent accuracy on the GalaxyMNIST test set. The CNN costs 8.228 mJ per inference, about 2.9 times more, but it gains the most from the vector unit, a 7.44 times energy reduction over its scalar build. The next step is to measure the CNN accuracy on the same labelled test set so the energy comparison can be paired with an accuracy comparison.

## Caveats

The absolute watts depend on the 22 nm, 100 MHz and fp 0.60 assumptions in the McPAT model. The ratios, both RVV versus scalar and CNN versus S4D, are robust to these choices and are the part to trust. We report only the in order RiscvMinorCPU results. An out of order O3 core was run but excluded, because its power estimate required many out of order statistics that the tooling had to fill in rather than measure. A SiFive U74 board run produced an empty statistics file and is also excluded. The CNN accuracy is not yet measured. Energy equals runtime dynamic power multiplied by execution time, with each build paired with its own time.